

Лабораторная работа №15

Алгоритм Кнута-Морриса-Пратта. STL (обзор)

Цель работы:

1. Изучить алгоритм Кнута-Морриса-Пратта (КМП) для эффективного поиска подстроки в строке
2. Реализовать построение префикс-функции для заданной строки
3. Реализовать поиск всех вхождений образца в текст с использованием КМП
4. Освоить основные контейнеры и алгоритмы стандартной библиотеки шаблонов (STL)
5. Научиться применять STL для решения практических задач

Теоретическое обоснование

1. Поиск подстроки: постановка задачи

Задача поиска подстроки (pattern matching): Даны текст T длины n и образец (pattern) P длины m . Найти все позиции в тексте, где начинается вхождение образца.

Алгоритм	Временная сложность	Память
Наивный	$O(n \times m)$	$O(1)$
КМП	$O(n + m)$	$O(m)$

2. Наивный алгоритм поиска

```
vector<int> naiveSearch(const string& text, const string& pattern) {  
    vector<int> positions;  
    int n = text.length();  
    int m = pattern.length();  
  
    for (int i = 0; i <= n - m; i++) {  
        bool found = true;  
        for (int j = 0; j < m; j++) {  
            if (text[i + j] != pattern[j]) {  
                found = false;  
                break;  
            }  
        }  
        if (found) positions.push_back(i);  
    }  
}
```

```

    }
    return positions;
}

```

Проблема: При несовпадении сдвигаемся на 1 позицию, теряя уже проверенную информацию.

3. Префикс-функция (Prefix Function)

Префикс-функция $\pi[i]$ - это длина наибольшего собственного префикса строки $s[0..i]$, который также является её суффиксом.

Визуализация префикс-функции для строки "ababaca":

Индекс: 0 1 2 3 4 5 6
 Строка: a b a b a c a
 π : 0 0 1 2 3 0 1

$\pi[0] = 0$ (префикс "a" - нет собственного префикса)
 $\pi[1] = 0$ ("ab" - префиксы: "a", суффиксы: "b" → нет совпадений)
 $\pi[2] = 1$ ("aba" - префикс "a" = суффикс "a")
 $\pi[3] = 2$ ("abab" - префикс "ab" = суффикс "ab")
 $\pi[4] = 3$ ("ababa" - префикс "aba" = суффикс "aba")
 $\pi[5] = 0$ ("ababac" - нет совпадений)
 $\pi[6] = 1$ ("ababaca" - префикс "a" = суффикс "a")

Алгоритм вычисления префикс-функции $O(n)$:

```

vector<int> computePrefixFunction(const string& pattern) {
    int m = pattern.length();
    vector<int> pi(m, 0);

    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j > 0 && pattern[i] != pattern[j]) {
            j = pi[j - 1];
        }
        if (pattern[i] == pattern[j]) {
            j++;
        }
        pi[i] = j;
    }

    return pi;
}

```

}

4. Алгоритм Кнута-Морриса-Пратта (КМП)

Принцип работы: Использует префикс-функцию для эффективного сдвига при несовпадении, не возвращаясь назад по тексту.

Визуализация работы КМП:

Текст: a b a b a b c a

Образец: a b a b c

Префикс-функция образца: [0,0,1,2,0]

Сравнение:

a b a b a b c a

a b a b c

↑ несовпадение ($b \neq c$)

По префикс-функции: для $j=4$, $\pi[3]=2 \rightarrow$ сдвигаем образец на $(4-2)=2$ позиции

a b a b a b c a

a b a b c

↑ продолжаем сравнение

Алгоритм поиска $O(n + m)$:

```
vector<int> kmpSearch(const string& text, const string& pattern) {  
    vector<int> positions;  
    int n = text.length();  
    int m = pattern.length();  
  
    if (m == 0) return positions;  
  
    vector<int> pi = computePrefixFunction(pattern);  
  
    int j = 0; // количество совпавших символов образца  
    for (int i = 0; i < n; i++) {  
        while (j > 0 && text[i] != pattern[j]) {  
            j = pi[j - 1];  
        }  
        if (text[i] == pattern[j]) {  
            j++;  
        }  
        if (j == m) {  
            positions.push_back(i - m + 1);  
        }  
    }  
}
```

```

        j = pi[j - 1];
    }
}

return positions;
}

```

5. Обзор стандартной библиотеки шаблонов (STL)

STL - это мощная библиотека C++, состоящая из контейнеров, итераторов, алгоритмов и функциональных объектов.

Контейнеры STL

Категория	Контейнеры	Описание
Последовательные	vector, list, deque, array	Линейное хранение элементов
Ассоциативные	set, map, multiset, multimap	Хранение по ключу (сбалансированное дерево)
Неупорядоченные	unordered_set, unordered_map	Хранение по ключу (хеш-таблица)
Адаптеры	stack, queue, priority_queue	Ограниченные интерфейсы

Итераторы

// Пример использования итераторов

```
vector<int> v = {1, 2, 3, 4, 5};
```

// Обычный итератор

```
for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << " ";
}
```

// Константный итератор

```
for (auto it = v.cbegin(); it != v.cend(); ++it) {
    // *it = 10; // Ошибка!
}
```

// Обратный итератор

```
for (auto it = v.rbegin(); it != v.rend(); ++it) {  
    cout << *it << " ";  
}
```

// Range-based for (C++11)

```
for (int x : v) {  
    cout << x << " ";  
}
```

Алгоритмы STL

cpp

```
#include <algorithm>
```

```
#include <numeric>
```

```
vector<int> v = {5, 2, 8, 1, 9, 3};
```

// Сортировка

```
sort(v.begin(), v.end());           // {1, 2, 3, 5, 8, 9}  
sort(v.begin(), v.end(), greater<int>()); // {9, 8, 5, 3, 2, 1}
```

// Поиск

```
auto it = find(v.begin(), v.end(), 5);    // ищет значение 5  
auto it2 = find_if(v.begin(), v.end(), [](int x) { return x > 5; });
```

// Копирование

```
vector<int> dest(v.size());  
copy(v.begin(), v.end(), dest.begin());
```

// Преобразование

```
transform(v.begin(), v.end(), v.begin(), [](int x) { return x * 2; });
```

// Подсчёт

```
int cnt = count(v.begin(), v.end(), 3);  
int cntIf = count_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
```

// Удаление (erase-remove idiom)

```
v.erase(remove(v.begin(), v.end(), 5), v.end());
```

// Минимум/максимум

```
int mn = *min_element(v.begin(), v.end());  
int mx = *max_element(v.begin(), v.end());
```

// Сумма (numeric)

```
int sum = accumulate(v.begin(), v.end(), 0);
```

Примеры выполнения практической части

Пример 1. Префикс-функция (базовый уровень)

Задача: Реализовать вычисление префикс-функции для заданной строки.

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

vector<int> computePrefixFunction(const string& s) {
    int n = s.length();
    vector<int> pi(n, 0);

    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) {
            j = pi[j - 1];
        }
        if (s[i] == s[j]) {
            j++;
        }
        pi[i] = j;
    }

    return pi;
}

int main() {
    string s = "ababaca";
    vector<int> pi = computePrefixFunction(s);

    cout << "Строка: " << s << endl;
    cout << "Префикс-функция: ";
    for (int x : pi) {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}
```

```
}
```

Ожидаемый вывод:

Строка: ababasa

Префикс-функция: 0 0 1 2 3 0 1

Пример 2. Алгоритм КМП (средний уровень)

Задача: Реализовать поиск всех вхождений образца в текст с помощью КМП.

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
using namespace std;
```

```
vector<int> computePrefixFunction(const string& pattern) {
```

```
    int m = pattern.length();
```

```
    vector<int> pi(m, 0);
```

```
    for (int i = 1; i < m; i++) {
```

```
        int j = pi[i - 1];
```

```
        while (j > 0 && pattern[i] != pattern[j]) {
```

```
            j = pi[j - 1];
```

```
        }
```

```
        if (pattern[i] == pattern[j]) {
```

```
            j++;
```

```
        }
```

```
        pi[i] = j;
```

```
    }
```

```
    return pi;
```

```
}
```

```
vector<int> kmpSearch(const string& text, const string& pattern) {
```

```
    vector<int> positions;
```

```
    int n = text.length();
```

```
    int m = pattern.length();
```

```
    if (m == 0) return positions;
```

```
    if (m > n) return positions;
```

```
    vector<int> pi = computePrefixFunction(pattern);
```

```

int j = 0;
for (int i = 0; i < n; i++) {
    while (j > 0 && text[i] != pattern[j]) {
        j = pi[j - 1];
    }
    if (text[i] == pattern[j]) {
        j++;
    }
    if (j == m) {
        positions.push_back(i - m + 1);
        j = pi[j - 1];
    }
}

return positions;
}

int main() {
    string text = "ababaababcabab";
    string pattern = "abab";

    vector<int> positions = kmpSearch(text, pattern);

    cout << "Текст: " << text << endl;
    cout << "Образец: " << pattern << endl;
    cout << "Вхождения на позициях: ";
    for (int pos : positions) {
        cout << pos << " ";
    }
    cout << endl;

    // Визуализация
    cout << "\nВизуализация:" << endl;
    cout << text << endl;
    for (int pos : positions) {
        for (int i = 0; i < pos; i++) cout << " ";
        cout << pattern << endl;
    }

    return 0;
}

```

Ожидаемый вывод:

Текст: ababaababscabab

Образец: abab

Вхождения на позициях: 0 2 6 10

Визуализация:

ababaababscabab

abab

abab

abab

abab

Пример 3. Сравнение наивного и КМП алгоритмов (продвинутый уровень)

Задача: Сравнить производительность наивного и КМП алгоритмов.

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include <chrono>
```

```
#include <random>
```

```
using namespace std;
```

```
using namespace chrono;
```

```
// Наивный алгоритм
```

```
vector<int> naiveSearch(const string& text, const string& pattern) {
```

```
    vector<int> positions;
```

```
    int n = text.length();
```

```
    int m = pattern.length();
```

```
    for (int i = 0; i <= n - m; i++) {
```

```
        bool found = true;
```

```
        for (int j = 0; j < m; j++) {
```

```
            if (text[i + j] != pattern[j]) {
```

```
                found = false;
```

```
                break;
```

```
            }
```

```
        }
```

```
        if (found) positions.push_back(i);
```

```
    }
```

```
    return positions;
```

```
}
```

```
// Префикс-функция
```

```
vector<int> computePrefixFunction(const string& pattern) {  
    int m = pattern.length();  
    vector<int> pi(m, 0);  
  
    for (int i = 1; i < m; i++) {  
        int j = pi[i - 1];  
        while (j > 0 && pattern[i] != pattern[j]) {  
            j = pi[j - 1];  
        }  
        if (pattern[i] == pattern[j]) {  
            j++;  
        }  
        pi[i] = j;  
    }  
    return pi;  
}
```

```
// Алгоритм КМП
```

```
vector<int> kmpSearch(const string& text, const string& pattern) {  
    vector<int> positions;  
    int n = text.length();  
    int m = pattern.length();  
  
    if (m == 0 || m > n) return positions;  
  
    vector<int> pi = computePrefixFunction(pattern);  
    int j = 0;  
  
    for (int i = 0; i < n; i++) {  
        while (j > 0 && text[i] != pattern[j]) {  
            j = pi[j - 1];  
        }  
        if (text[i] == pattern[j]) {  
            j++;  
        }  
        if (j == m) {  
            positions.push_back(i - m + 1);  
            j = pi[j - 1];  
        }  
    }  
}
```

```

    }
    return positions;
}

// Генерация случайной строки
string generateRandomString(int length, int alphabetSize = 2) {
    string result;
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(0, alphabetSize - 1);

    for (int i = 0; i < length; i++) {
        result += 'a' + dis(gen);
    }
    return result;
}

int main() {
    cout << "=== Сравнение алгоритмов поиска подстроки ===" << endl;

    vector<int> lengths = {1000, 2000, 5000, 10000, 20000};

    cout << "\nТекст: случайный алфавит (a,b)" << endl;
    cout << "Образец: \"ababbaba\"" << endl;
    cout << "\nДлина текста\tНаивный (мкс)\tКМП (мкс)" << endl;

    string pattern = "ababbaba";

    for (int n : lengths) {
        string text = generateRandomString(n);

        // Наивный алгоритм
        auto start = high_resolution_clock::now();
        auto naiveResult = naiveSearch(text, pattern);
        auto end = high_resolution_clock::now();
        auto naiveTime = duration_cast<microseconds>(end - start).count();

        // Алгоритм КМП
        start = high_resolution_clock::now();
        auto kmpResult = kmpSearch(text, pattern);
        end = high_resolution_clock::now();
        auto kmpTime = duration_cast<microseconds>(end - start).count();
    }
}

```

```

        cout << n << "\t\t" << naiveTime << "\t\t" << kmpTime << endl;
    }

    return 0;
}

```

Пример 4. Применение STL (обзор)

Задача: Продемонстрировать основные возможности STL.

```

#include <iostream>
#include <vector>
#include <list>
#include <deque>
#include <set>
#include <map>
#include <unordered_set>
#include <unordered_map>
#include <algorithm>
#include <numeric>
#include <string>

using namespace std;

int main() {
    cout << "=== STL: контейнеры и алгоритмы ===" << endl;

    // 1. vector - динамический массив
    cout << "\n1. vector:" << endl;
    vector<int> v = {5, 2, 8, 1, 9, 3};
    cout << "Исходный: ";
    for (int x : v) cout << x << " ";
    cout << endl;

    sort(v.begin(), v.end());
    cout << "Отсортированный: ";
    for (int x : v) cout << x << " ";
    cout << endl;

    v.push_back(10);
    cout << "После push_back(10): ";
    for (int x : v) cout << x << " ";
    cout << endl;
}

```

// 2. list - двусвязный список

```
cout << "\n2. list:" << endl;
list<int> lst = {1, 2, 3, 4, 5};
lst.push_front(0);
lst.push_back(6);
cout << "Элементы: ";
for (int x : lst) cout << x << " ";
cout << endl;
```

// 3. set - упорядоченное множество

```
cout << "\n3. set:" << endl;
set<int> s = {3, 1, 4, 1, 5, 9, 2, 6};
cout << "Уникальные элементы: ";
for (int x : s) cout << x << " ";
cout << endl;
cout << "Есть ли 5? " << (s.count(5) ? "да" : "нет") << endl;
```

// 4. map - словарь

```
cout << "\n4. map:" << endl;
map<string, int> ages = {"Alice", 25}, {"Bob", 30}, {"Charlie", 35};
ages["David"] = 28;
for (const auto& [name, age] : ages) {
    cout << name << ": " << age << endl;
}
```

// 5. unordered_set - хеш-множество

```
cout << "\n5. unordered_set:" << endl;
unordered_set<string> words = {"hello", "world", "c++", "stl"};
words.insert("algorithm");
for (const string& w : words) {
    cout << w << " ";
}
cout << endl;
```

// 6. Алгоритмы STL

```
cout << "\n6. Алгоритмы STL:" << endl;
vector<int> nums = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

// find

```
auto it = find(nums.begin(), nums.end(), 7);
cout << "find(7): позиция " << (it - nums.begin()) << endl;
```

```

// count_if
int evenCount = count_if(nums.begin(), nums.end(), [](int x) { return x %
2 == 0; });
cout << "Количество чётных: " << evenCount << endl;

// transform
vector<int> squares(nums.size());
transform(nums.begin(), nums.end(), squares.begin(), [](int x) { return x *
x });

cout << "Квадраты: ";
for (int x : squares) cout << x << " ";
cout << endl;

// accumulate
int sum = accumulate(nums.begin(), nums.end(), 0);
cout << "Сумма: " << sum << endl;

// reverse
reverse(nums.begin(), nums.end());
cout << "После reverse: ";
for (int x : nums) cout << x << " ";
cout << endl;

return 0;
}

```

Индивидуальные задания

Часть 1. Алгоритм КМП

Каждый вариант содержит задание реализовать алгоритм КМП с указанными особенностями:

№ вар.	Особенности	Дополнительное задание
1	Префикс-функция	Вывести все вхождения
2	Префикс-функция	Вывести первое вхождение
3	КМП поиск	Вывести все вхождения
4	КМП поиск	Вывести первое вхождение

№ вар.	Особенности	Дополнительное задание
5	КМП поиск	Подсчитать количество вхождений
6	КМП поиск	Поиск с перекрытием
7	КМП поиск	Поиск без перекрытия
8	КМП поиск	Регистронезависимый поиск
9	КМП поиск	Поиск для кириллицы
10	КМП поиск	Поиск всех образцов из списка
11	КМП поиск	Замена всех вхождений
12	КМП поиск	Наибольшее совпадение с префиксом
13	КМП поиск	Наибольшее совпадение с суффиксом
14	КМП поиск	Поиск в файле
15	КМП поиск	Сравнение с наивным алгоритмом

Часть 2. STL (обзор)

Каждый вариант содержит задание с использованием STL для решения задачи:

№ вар.	Задача	Требуемые контейнеры/алгоритмы
1	Подсчёт частоты слов в тексте	map<string, int>, istringstream
2	Удаление дубликатов из массива	vector, sort, unique
3	Проверка на анаграмму	sort, string

№ вар.	Задача	Требуемые контейнеры/алгоритмы
4	Нахождение пересечения двух множеств	set, set_intersection
5	Нахождение объединения двух множеств	set, set_union
6	Сортировка студентов по баллам	vector, sort, pair
7	Поиск ближайшего числа в отсортированном массиве	vector, lower_bound
8	Подсчёт уникальных элементов	unordered_set
9	Частотный словарь символов	unordered_map<char, int>
10	Топ К самых частых слов	map, vector, sort
11	Проверка палиндрома	string, reverse, equal
12	Циклический сдвиг	deque, rotate
13	Объединение двух отсортированных списков	list, merge
14	Фильтрация по условию	vector, copy_if
15	Группировка элементов по признаку	map, partition

Часть 3. Применение

Задание для всех вариантов:

1. Реализовать программу, которая:
2. Считывает текст из файла или с клавиатуры
3. Считывает образец для поиска
4. Используя КМП, находит все вхождения образца
5. Выводит позиции вхождений

6. Используя STL, выполняет дополнительную обработку (по варианту)

Контрольные вопросы

1. В чём заключается идея алгоритма Кнута-Морриса-Пратта?
2. Что такое префикс-функция и как она вычисляется?
3. Какова временная сложность алгоритма КМП? Почему?
4. Как работает восстановление по префикс-функции при несовпадении?
5. В чём преимущество КМП перед наивным алгоритмом?
6. Какие контейнеры STL вы знаете? Какой контейнер когда лучше использовать?
7. В чём разница между vector и list?
8. В чём разница между set и unordered_set?
9. В чём разница между map и unordered_map?
10. Как работают итераторы в STL?
11. Какие алгоритмы STL вы знаете? Приведите примеры.
12. Что такое лямбда-выражения в C++? Как они применяются с алгоритмами STL?
13. Как удалить элементы из контейнера во время итерации?
14. Что такое идиома erase-remove?
15. Как можно изменить порядок обхода контейнера?

Требования к отчёту

Титульный лист

Цель работы

Теоретическая часть (КМП, префикс-функция, STL)

Постановка задачи (согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода)

Анализ сложности алгоритма КМП

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

№	Критерий	Баллы
1	Префикс-функция	20

№	Критерий	Баллы
1.1	Реализация алгоритма	10
1.2	Корректность работы	10
2	Алгоритм КМП	35
2.1	Реализация поиска	15
2.2	Корректность работы	15
2.3	Дополнительные возможности (по варианту)	5
3	STL (обзор)	30
3.1	Корректное использование контейнеров	15
3.2	Корректное использование алгоритмов	15
4	Оформление и отчёт	15
4.1	Код отформатирован, есть комментарии	5
4.2	Есть ответы на контрольные вопросы	5
4.3	Ссылка на Git-репозиторий	5
Итого		100

Штрафные баллы

Нарушение	Штраф
Неправильное вычисление префикс-функции	-15
Неправильная работа КМП	-15

Нарушение	Штраф
Неэффективное использование STL	-10
Нет ответов на контрольные вопросы	-10
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60